

Analiza Experimentală a Algoritmilor de Sortare

Justificarea Metodologiei, Timp și Memorie

Costin Anica

Martie 2026

1 Introducere

Acest proiect analizează performanța algoritmilor **BUBBLE**, **SELECT**, **INSERT**, **MERGE** și **QUICK SORT** pe eșantioane de date de la 10 la 10.000 de elemente.

2 Metodologia Scalării și Sustenabilitatea Testării

O componentă cheie a experimentului a fost adaptarea numărului de rulări în funcție de dimensiunea N :

- **Faza Micro ($N \leq 100$):** S-au efectuat **1.000.000** de repetări.
- **Faza Macro ($N \geq 1.000$):** S-a trecut la o singură rulare per tip de date.

Justificare teoretică: Dacă am fi menținut pragul de 1.000.000 de repetări pentru $N = 10.000$, unde **Bubble Sort** a scos un timp de ~ 437 ms, experimentul ar fi durat peste **121 de ore**, ceea ce demonstrează ineficiența practică a algoritmilor $O(n^2)$.

3 Rezultate Experimentale: Timp și Memorie

3.1 Timp Total Faza Micro (1.000.000 repetări)

În această etapă, s-a măsurat timpul cumulat pentru un milion de rulări pe liste mici, incluzând complexitatea teoretică a timpului pentru fiecare metodă:

Algoritm	Complexitate	N=10 (s)	N=20 (s)	N=50 (s)	N=100 (s)
BUBBLE	$O(n^2)$	0.549 s	1.590 s	9.600 s	40.378 s
SELECT	$O(n^2)$	0.320 s	0.910 s	5.468 s	20.250 s
INSERT	$O(n^2)$	0.330 s	0.735 s	3.366 s	12.941 s
MERGE	$O(n \log n)$	2.145 s	4.580 s	12.629 s	27.325 s
QUICK	$O(n \log n)$	0.314 s	0.496 s	1.392 s	3.496 s

3.2 Evoluția Timpului în Faza Medie și Macro (ms)

Măsurători pentru o singură rulare, evidențiind scalabilitatea algoritmilor pe măsură ce N crește:

N	BUBBLE	SELECT	MERGE	QUICK
200	0.152 ms	0.077 ms	0.065 ms	0.016 ms
1000	4.091 ms	1.697 ms	0.353 ms	0.092 ms
10000	437.39 ms	164.87 ms	3.956 ms	1.162 ms

3.3 Consumul de Memorie Auxiliară

Tabelul următor estimează spațiul suplimentar utilizat de fiecare metodă:

Algoritm	Complexitate Memorie	Consum N=10.000
BUBBLE	$O(1)$	$\sim$ bytes (constante)
SELECT	$O(1)$	$\sim$ bytes (constante)
INSERT	$O(1)$	$\sim$ bytes (constante)
MERGE	$O(n)$	$\sim$ 40.000 bytes
QUICK	$O(\log n)$	$\sim$ 128 bytes (stivă)

4 Analiza Comportamentului Neobișnuit

4.1 Eficiența Cache-ului vs. Teorie

Deși Quick Sort și Merge Sort sunt ambele $O(n \log n)$, Quick Sort este constant mai rapid. Motivul este **localitatea referinței** (Cache Locality). Quick Sort lucrează *in-place*, în timp ce Merge Sort provoacă **Cache Misses** prin copierea în vectori auxiliari.

4.2 Alocarea Dinamică (Merge Sort)

Timpul mare al Merge Sort la $N = 10$ confirmă că **managementul memoriei Heap** (new/delete) este mai costisitor decât sortarea propriu-zisă pe liste mici.

5 Concluzii

Rezultatele confirmă modelele teoretice, dar subliniază importanța arhitecturii hardware (RAM/Cache) în performanța reală.

6 Resurse

Repository: <https://github.com/costi7799/Sorting-Algorithms-Analysis>